



DEEP LEARNING AND NATURAL LANGUAGE PROCESSING

LECTURE 8: TRANSFORMER ARCHITECTURE AND ATTENTION MECHANISMS

Dr. Adewale Akinfaderin

SEAS 8520-D6A · Summer 2026



Table of Contents

1 From Sequences to Attention

- ▶ From Sequences to Attention
- ▶ The Attention Mechanism
- ▶ Self-Attention and the Attention Head
- ▶ The Transformer Block
- ▶ Architectures: GPT, BERT, and Encoder–Decoder
- ▶ Scale, Efficiency, and Limits
- ▶ A Case Study: Does Architecture Win?
- ▶ Wrap-Up

Where we left off. Words became dense, learned **embeddings**, and meaning came from context (the distributional hypothesis). The catch: a static embedding gives *one* vector per word, no matter the sentence. Today we make representations **contextual** using one mechanism, **attention**, and end able to read the architecture behind BERT and GPT.

1. **From sequences to attention:** recurrent networks, their bottleneck, and why we replaced them.
2. **The attention mechanism:** a weighted sum that builds contextual meaning.
3. **Self-attention and the attention head:** queries, keys, values, scaling, multi-head, the matrix view.
4. **The transformer block:** residual stream, layer norm, feedforward, positional and token embeddings.
5. **Architectures:** decoder-only (GPT), encoder-only (BERT), encoder-decoder, and pretrain/fine-tune.
6. **Scale, efficiency, limits:** scaling laws, the KV cache, and what transformers still get wrong.

A sentence is an **ordered** list of tokens. Order carries meaning:

“the dog chased the cat” $\neq$ *“the cat chased the dog”*

- Bag-of-words throws order away; it cannot tell these apart.
- We want a model whose representation of a token **depends on its neighbors**.
- First attempt (historically): carry information forward one step at a time with a **recurrent** network.

Every network we have used so far, a dense classifier or a convnet, is a **feedforward** network: it processes each input independently, with **no state kept between inputs**. Show it a sentence and it must see the whole thing at once, as one fixed-size blob.

- An **MLP** needs a fixed-length input. A sentence has variable length, and flattening it discards word order.
- A **CNN** slides a small window over the text, so it captures *local* patterns but has no mechanism to carry information across a long sentence.
- Yet when *you* read this sentence, you process it word by word while **keeping a memory** of what came before. Meaning accumulates.

A **recurrent neural network** adopts exactly that principle: it walks through the sequence one token at a time and maintains an internal **state** that carries information about everything seen so far.

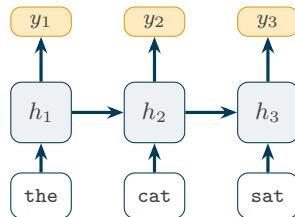
The RNN is really just a **loop**. At step t it combines the new token $\mathbf{x}_t$ with the previous state $\mathbf{h}_{t-1}$ to produce an updated state $\mathbf{h}_t$:

$$\mathbf{h}_t = f(\mathbf{W} \mathbf{x}_t + \mathbf{U} \mathbf{h}_{t-1} + \mathbf{b})$$

Reading it piece by piece:

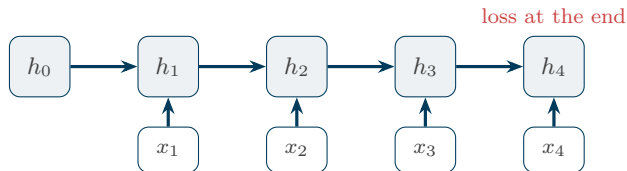
- $\mathbf{W} \mathbf{x}_t$: what the **current word** contributes.
- $\mathbf{U} \mathbf{h}_{t-1}$: what the **memory of the past** contributes.
- f is a squashing nonlinearity (tanh); the same $\mathbf{W}, \mathbf{U}, \mathbf{b}$ are **reused every step**.

So $\mathbf{h}_t$ summarizes the prefix $x_1 \dots x_t$, and an output reads off it: $\mathbf{y}_t = g(\mathbf{V} \mathbf{h}_t + \mathbf{c})$.



The state (blue) flows left to right, carrying memory forward; an output (gold) is read at each step.

The recurrence is one cell applied repeatedly. **Unrolling** it across the sequence turns the loop into a deep feedforward graph that shares weights:



Training runs the forward pass over the whole unrolled chain, then backpropagates the loss through every step. This is **backpropagation through time** (BPTT), and its depth equals the sequence length.

A language model predicts the next token from the prefix. An RNN does this naturally by emitting a distribution at every step:

$$P(x_{t+1} \mid x_1 \dots x_t) = \text{softmax}(\mathbf{V} \mathbf{h}_t + \mathbf{c}).$$

- Train by **teacher forcing**: feed the true token x_t as input and score the prediction of x_{t+1} .
- The loss is the average cross-entropy over the sequence, which is the **perplexity** objective.
- Generate text by sampling one token, feeding it back in, and repeating.

This left-to-right factorization is exactly what GPT models keep, even after recurrence is replaced by attention.

Recurrence solves exactly the problems that stopped the MLP and CNN from handling text well:

- **Handles any length.** The same cell is applied in a loop, so one model reads a five-word sentence or a five-hundred-word document without changing its parameters.
- **Respects word order.** Because the state is built up token by token, swapping two words produces different hidden states, so “*dog bites man*” and “*man bites dog*” are encoded differently.
- **Parameter efficient.** One shared set of weights covers every position, instead of a separate parameter for each slot in the sentence.

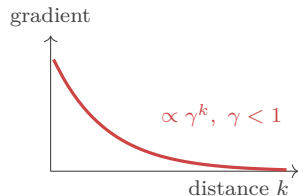
This gives two natural ways to use an RNN: read out the **final** state $\mathbf{h}_T$ as a summary of the whole sequence for **classification** (sentiment, topic), or read out **every** state $\mathbf{h}_t$ for **sequence labeling**, which assigns one tag per token (part-of-speech, named entities).

In principle $\mathbf{h}_t$ remembers the whole prefix. In practice a plain RNN struggles to connect far-apart words, because **training** pushes an error signal backward through every step between them.

To link token t to a word k steps back, the gradient is multiplied by the recurrent weight about k times, by a factor γ each time. Two failure modes:

- If $\gamma < 1$: the signal shrinks toward 0, the **vanishing gradient**, so the model *forgets* the distant past.
- If $\gamma > 1$: the signal blows up toward ∞ , the **exploding gradient**, making training unstable.

Clipping tames explosion; vanishing needs a new **architecture** (the LSTM, next).



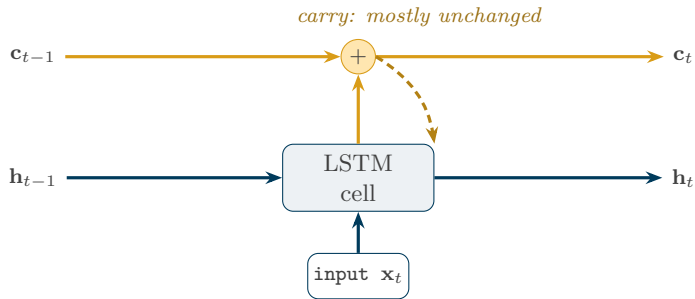
The influence of a token k steps away decays exponentially, so early words barely affect learning at the end.

The **Long Short-Term Memory** cell adds a separate **cell state** $\mathbf{c}_t$ with learned **gates** that decide what to keep, forget, and output (Hochreiter & Schmidhuber, 1997):

- The **forget gate** sets how much of the old cell state to erase.
- The **input gate** sets how much new information to write.
- The **output gate** sets how much of the cell to expose as $\mathbf{h}_t$.

Because the cell state has an **additive** path through time, gradients survive much longer than in a plain RNN, which is what makes long-range learning possible.

The LSTM's key move is to add a second track running **beside** the ordinary state: a **cell state** $\mathbf{c}_t$ that acts like a highway. Information travels along it across many steps almost untouched, instead of being squeezed through a nonlinearity at every step.



At each step the cell reads the highway, writes a small learned update onto it (the +), and passes the exposed part out as $\mathbf{h}_t$. Because updates are *added* onto the highway instead of multiplying through it, gradients survive far longer. **Gates** (next slide) decide how much to erase, write, and expose.

Each **gate** is a small layer that outputs numbers between 0 and 1 (a sigmoid σ), acting like a dial from “block completely” to “let through fully.” Three gates control the memory highway:

$$\mathbf{f}_t = \sigma(\mathbf{W}_f[\mathbf{h}_{t-1}, \mathbf{x}_t] + \mathbf{b}_f) \quad \text{forget}$$

$$\mathbf{i}_t = \sigma(\mathbf{W}_i[\mathbf{h}_{t-1}, \mathbf{x}_t] + \mathbf{b}_i) \quad \text{input}$$

$$\mathbf{o}_t = \sigma(\mathbf{W}_o[\mathbf{h}_{t-1}, \mathbf{x}_t] + \mathbf{b}_o) \quad \text{output}$$

$$\tilde{\mathbf{c}}_t = \tanh(\mathbf{W}_c[\mathbf{h}_{t-1}, \mathbf{x}_t] + \mathbf{b}_c) \quad \text{candidate}$$

$$\mathbf{c}_t = \mathbf{f}_t \odot \mathbf{c}_{t-1} + \mathbf{i}_t \odot \tilde{\mathbf{c}}_t$$

$$\mathbf{h}_t = \mathbf{o}_t \odot \tanh(\mathbf{c}_t)$$

Read it as a memory update:

- **Forget $\mathbf{f}_t$:** how much of the old memory $\mathbf{c}_{t-1}$ to keep.
- **Input $\mathbf{i}_t$:** how much of the new candidate $\tilde{\mathbf{c}}_t$ to write in.
- **Output $\mathbf{o}_t$:** how much of the memory to expose as this step’s hidden state $\mathbf{h}_t$.

The takeaway

The new memory is $\mathbf{c}_t = (\text{keep}) \cdot \text{old} + (\text{write}) \cdot \text{new}$. That **addition** is the whole point: when the forget gate stays near 1, memory passes through unchanged and gradients flow back untouched, so the LSTM can remember information across hundreds of steps.

Two Useful Variants: GRUs and Bidirectional RNNs

1 From Sequences to Attention

The LSTM works, but it is heavy (three gates, an extra state) and it still only reads left to right. Two variants address these:

GRU: a lighter gate.

The Gated Recurrent Unit keeps the gating idea but merges the cell and hidden states and uses only *two* gates (Cho et al., 2014).

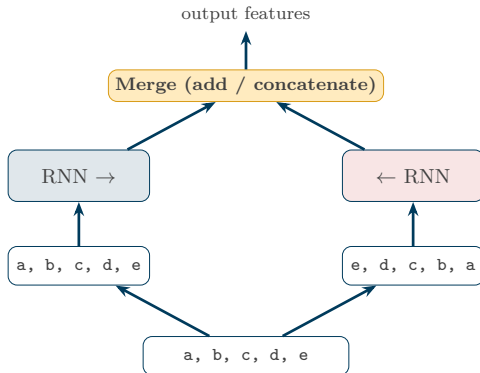
- *Why*: fewer parameters, faster to train, less data to fit, yet often matches the LSTM.
- A common default when the LSTM feels too costly.

Bidirectional: read both ways.

A left-to-right RNN only knows the *past*. But to tag a word, the words *after* it also help.

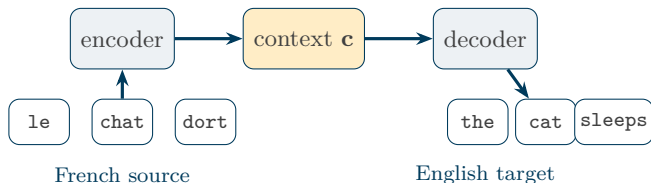
- *Why*: in “I **book** a flight” vs. “read a **book**”, the following words settle the meaning.
- Run one RNN forward and one backward, then merge, so each position sees both sides. Standard for tagging and encoding, but it cannot drive left-to-right generation (it needs the whole sentence up front).

Feed the sequence to one RNN in order and a copy of it **reversed** to a second RNN, then merge, so each position is informed by both directions:



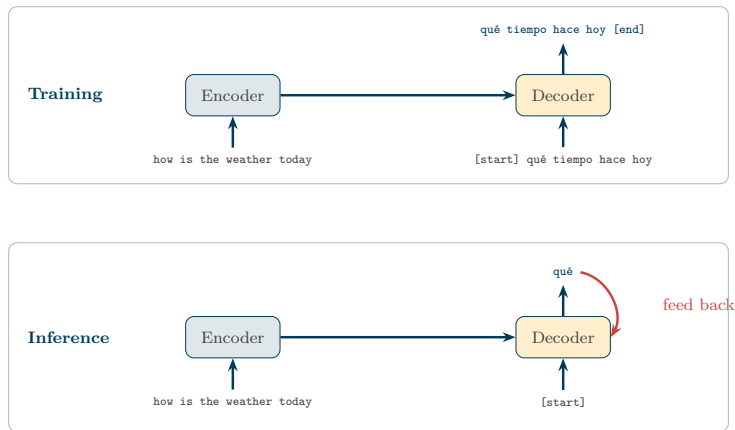
This helps tagging and encoding, but it cannot drive left-to-right generation, since the backward pass needs the whole sequence up front.

So far one RNN produced one output per input token. But **machine translation** maps a whole sentence to a *different* sentence, often of a different length and word order. The **encoder-decoder** (Sutskever et al. 2014; Cho et al. 2014) splits this into two RNNs. Example: translate French to English.



The **encoder** RNN reads the French words and compresses them into a single fixed-length **context vector c** , its final hidden state. The **decoder** RNN starts from c and generates the English words one at a time. Nothing forces the two sentences to have the same length.

Same **machine-translation** task, now English→Spanish, in its two modes:

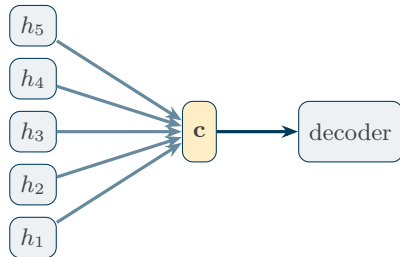


During training the decoder sees the whole target shifted by one (teacher forcing); at inference it emits one token, feeds it back, and repeats.

Forcing the entire source sentence through **one fixed vector** is a severe information bottleneck.

- Long sentences overflow the vector; early words get crushed.
- The decoder cannot “look back” at specific source words.

The fix that started it all: let the decoder, at each step, form a *weighted combination* of **all** encoder states instead of just the last one.



all states squeezed
into one **c**

Bahdanau Attention: The Prequel to Transformers

1 From Sequences to Attention

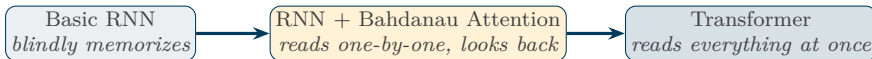
Attention did not *replace* RNNs. In 2014, Bahdanau, Cho, & Bengio added it **on top of** an RNN translator, and that add-on is the missing link that led straight to the transformer.

The idea (a magnifying glass). Rather than rely on one crushed context vector, let the decoder *look back* at every source word and focus on the relevant ones at each step:

- Score each source state $\mathbf{h}_j$, softmax, take a weighted sum: a **fresh context vector per output word**.
- The model **learns where to look**; long-sentence quality jumped immediately.

An analogy. A basic RNN is a student memorizing a 30-word sentence in one glance, then translating from memory, forgetting the start. With attention, that same student may **keep the book open** and point at relevant words while translating.

The breakthrough. If attention does the real work, do we need the slow RNN at all? In 2017: no. Use *only* attention, all words at once.



1. No parallelism.

$\mathbf{h}_t$ needs $\mathbf{h}_{t-1}$, so computation is inherently **sequential**. We cannot use a GPU's parallelism across the sequence during training.

2. Distance still hurts.

Even with LSTMs, information between two tokens must pass through every step between them. Path length grows with distance.

The transformer's bet

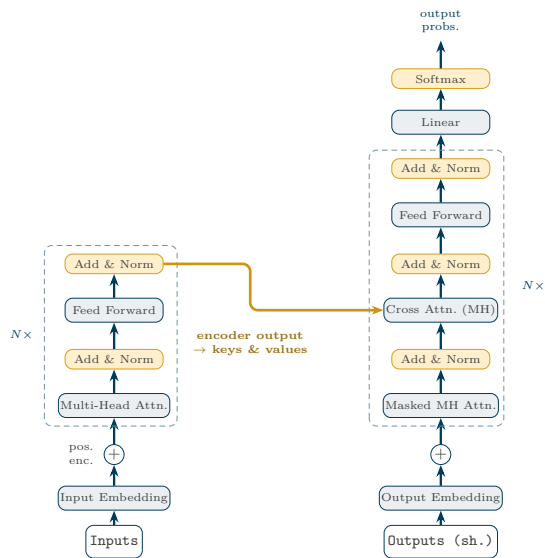
Compute the relationship between *every* pair of tokens **directly** and **in parallel**. Path length between any two tokens becomes **one**.

The 2017 paper introduced the **transformer**: an architecture built *entirely* from attention and feedforward layers, with **no recurrence and no convolution** (Vaswani et al., 2017). Removing the RNN unlocked three wins:

- **Full parallelism.** All positions are processed at once, using a GPU’s parallelism instead of marching token by token.
- **Constant path length.** Any two tokens interact *directly*, so distant words are as easy to relate as neighbors.
- **Scales beautifully.** The same design now powers state-of-the-art models for language, vision, and audio.



The paper that started the transformer era.



The original design has **two stacks** (Vaswani et al., 2017):

- **Encoder** (left): maps the input to contextual vectors, N times.
- **Decoder** (right): generates left to right using **masked** self-attention plus **cross-attention**.
- The **gold arrow** feeds the encoder output into cross-attention as its **keys and values**.

Every sublayer has a residual **Add & Norm**. Modern models often keep one stack: **BERT** (encoder) or **GPT** (decoder).



Table of Contents

2 The Attention Mechanism

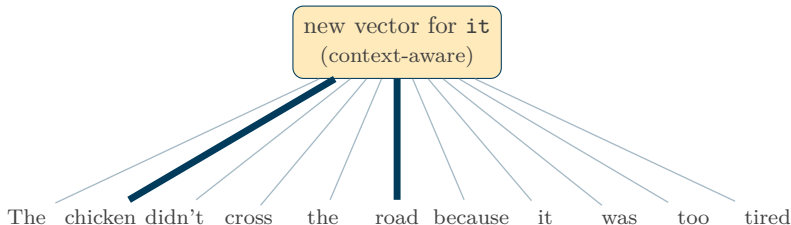
- ▶ From Sequences to Attention
- ▶ **The Attention Mechanism**
- ▶ Self-Attention and the Attention Head
- ▶ The Transformer Block
- ▶ Architectures: GPT, BERT, and Encoder–Decoder
- ▶ Scale, Efficiency, and Limits
- ▶ A Case Study: Does Architecture Win?
- ▶ Wrap-Up

A static embedding gives the word *it* one fixed vector. But its meaning depends on context:

- *The chicken didn't cross the road because **it** was too tired.* (*it* = chicken)
- *The chicken didn't cross the road because **it** was too wide.* (*it* = road)

Attention builds a contextual representation of *it* by pulling in information from the words it depends on. The model must *learn* that here those words are *chicken* and *road*.

To build a *contextual* vector for **it**, self-attention forms a **weighted blend of every word** in the sentence. Below, the thickness of each arrow is the weight the model assigns to that word. It has *learned* to put most weight on **chicken** and **road**, the two candidates **it** could refer to.



Thick arrow = high attention weight. The output replaces the generic embedding of **it** with a blend dominated by the words it actually depends on.

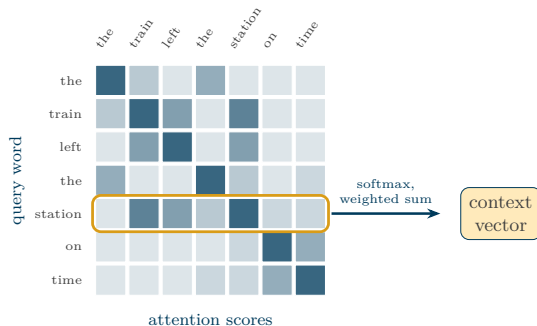
Strip away the machinery. At heart, the attention output $\mathbf{a}_i$ for token i is just a **weighted sum** of the other tokens' representations:

$$\mathbf{a}_i = \sum_{j \leq i} \alpha_{ij} \mathbf{x}_j$$

- α_{ij} says how much token j should contribute to token i .
- The weights are non-negative and sum to one (a distribution).
- For **causal** language modeling we sum only over $j \leq i$ (no peeking ahead).

Everything else is detail about *how we compute the weights* and *what we sum*.

The full attention matrix for “*the train left the station on time.*” Each **row** is a word acting as the *query*; each **column** is a candidate word; a **brighter cell** means the row word attends more to that column word. We spotlight the **station** row:



Reading that row, **station** attends most to *train* and *left*. Softmaxing the row and taking the weighted sum gives **station** a **context vector** that blends in those words, capturing that this is the station a train left from.

Computing the Weights: Why the Dot Product?

2 The Attention Mechanism

We need one number per word pair: *how relevant* is word j to word i ? Related meanings already sit close in embedding space, so **similarity** is exactly that signal. The cheapest measure is the **dot product**, $\mathbf{x}_i \cdot \mathbf{x}_j = \|\mathbf{x}_i\| \|\mathbf{x}_j\| \cos \theta$:

- Same direction ($\cos \theta \approx 1$): large positive score, highly related. Perpendicular: near 0. Opposing: negative.
- It is just multiply-and-add, so scoring all pairs is one fast matrix product on a GPU.

A raw score is unbounded, so we squash word i 's scores into weights that are positive and sum to 1 with a **softmax**:

$$\alpha_{ij} = \frac{\exp(\text{score}_{ij})}{\sum_{k \leq i} \exp(\text{score}_{ik})}$$

Let us run the whole recipe once, by hand, for the **third word** of a sentence. **Goal:** turn its plain embedding $\mathbf{x}_3$ into a context-aware vector $\mathbf{a}_3$ that has absorbed information from words 1–3.

1. **Score** how relevant each word is to word 3, using dot products: $\mathbf{x}_3 \cdot \mathbf{x}_1$, $\mathbf{x}_3 \cdot \mathbf{x}_2$, $\mathbf{x}_3 \cdot \mathbf{x}_3$.
2. **Normalize** the three scores with a softmax $\rightarrow \alpha_{31}, \alpha_{32}, \alpha_{33}$, which are non-negative and sum to 1.
3. **Blend:** take the weighted sum $\mathbf{a}_3 = \alpha_{31}\mathbf{x}_1 + \alpha_{32}\mathbf{x}_2 + \alpha_{33}\mathbf{x}_3$.

That is the entire mechanism: *score*, *normalize*, *blend*. A word usually attends strongly to *itself* ($\mathbf{x}_3 \cdot \mathbf{x}_3$ is large), but the softmax also spreads weight onto genuinely related context words.

So far every word used its raw vector $\mathbf{x}_i$ for everything. That is limiting: a word may want to *ask* for one thing yet *offer* something else. So the head gives each word three separate, learned views. Think of a **search engine**:

Query $\mathbf{q}_i = \mathbf{x}_i \mathbf{W}^Q$

What the current word is *looking for*, like the text you type into a search box.

Key $\mathbf{k}_j = \mathbf{x}_j \mathbf{W}^K$

What each other word *advertises*, like a page's title that a search matches against.

Value $\mathbf{v}_j = \mathbf{x}_j \mathbf{W}^V$

The *content* actually returned and summed if that word is attended to.

Soft dictionary lookup

Match the query against every key to get relevance scores; softmax them; use the result to weight the values. Unlike an ordinary lookup that returns one exact hit, attention returns a **blend** of values, weighted by how well each key matched.

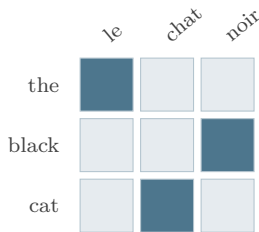
The dot product is the most common way to score query–key similarity, and several alternatives exist:

Name	Score($\mathbf{q}, \mathbf{k}$)	Note
Dot product	$\mathbf{q} \cdot \mathbf{k}$	cheap, used by transformers
Scaled dot	$\mathbf{q} \cdot \mathbf{k} / \sqrt{d_k}$	the transformer's choice
Additive	$\mathbf{w}^\top \tanh(\mathbf{W}_q \mathbf{q} + \mathbf{W}_k \mathbf{k})$	Bahdanau (2015)

Here d_k is the key-vector dimension; dividing by $\sqrt{d_k}$ keeps scores from growing as that dimension increases (next section). Additive attention uses a small network; scaled dot-product is faster on hardware because it is a single matrix multiply. Both learn *where to look*.

When a translation model attends, the weights form a soft **alignment** between source and target words. Here English *the black cat* aligns to French *le chat noir*:

- Each target word draws on the relevant source words.
- *black/noir* swap order, so that weight sits off the diagonal.
- The alignment is *learned*, never annotated.



Rows: English target. Columns: French source.

Darker = higher weight.



Table of Contents

3 Self-Attention and the Attention Head

- ▶ From Sequences to Attention
- ▶ The Attention Mechanism
- ▶ Self-Attention and the Attention Head
- ▶ The Transformer Block
- ▶ Architectures: GPT, BERT, and Encoder–Decoder
- ▶ Scale, Efficiency, and Limits
- ▶ A Case Study: Does Architecture Win?
- ▶ Wrap-Up

Every token arrives as a vector $\mathbf{x}_i$ of size d . Three *learned* matrices turn each token into three different vectors, one for each job it plays:

$$\underbrace{\mathbf{q}_i = \mathbf{x}_i \mathbf{W}^Q}$$

query: what am I looking for?

$$\underbrace{\mathbf{k}_j = \mathbf{x}_j \mathbf{W}^K}$$

key: what do I offer?

$$\underbrace{\mathbf{v}_j = \mathbf{x}_j \mathbf{W}^V}$$

value: what content do I pass on?

- $\mathbf{W}^Q, \mathbf{W}^K$ have shape $[d \times d_k]$; $\mathbf{W}^V$ has shape $[d \times d_v]$. Here d_k is the query/key dimension.
- So $\mathbf{q}_i$ and $\mathbf{k}_j$ live in the same d_k -space and can be compared; $\mathbf{v}_j$ carries the content that will actually be mixed.
- The same three matrices are applied at *every* position i, j .

With the three roles in hand, the head computes the output for token i in four steps:

$$\text{score}(\mathbf{x}_i, \mathbf{x}_j) = \frac{\mathbf{q}_i \cdot \mathbf{k}_j}{\sqrt{d_k}} \quad (1) \text{ how relevant is } j \text{ to } i$$

$$\alpha_{ij} = \text{softmax}_{j \leq i}(\text{score}(\mathbf{x}_i, \mathbf{x}_j)) \quad (2) \text{ weights that sum to 1}$$

$$\text{head}_i = \sum_{j \leq i} \alpha_{ij} \mathbf{v}_j \quad (3) \text{ blend the values}$$

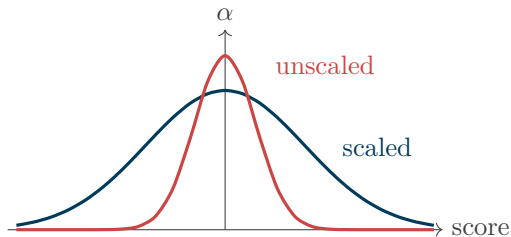
$$\mathbf{a}_i = \text{head}_i \mathbf{W}^O \quad (4) \text{ project back to size } d$$

The dot product $\mathbf{q}_i \cdot \mathbf{k}_j$ is a single number: how well token i 's query matches token j 's key.

The causal condition $j \leq i$ blocks the future. $\mathbf{W}^O$ ($[d_v \times d]$) restores the model width, so $\mathbf{x}_i$ and $\mathbf{a}_i$ are both $[1 \times d]$ and heads stack.

The score is a dot product of two d_k -dimensional vectors, where d_k is the **query/key dimension**. It sums d_k products, so if the entries are roughly independent with unit variance, the score has variance $\approx d_k$: bigger d_k means bigger, more spread-out scores.

- Large $d_k \Rightarrow$ large scores $\Rightarrow$ softmax saturates (one weight $\rightarrow 1$, rest $\rightarrow 0$).
- A saturated softmax has **tiny gradients**; learning stalls.
- Dividing by $\sqrt{d_k}$ rescales the variance back to ≈ 1 .



Scaled (blue) stays spread and differentiable;
unscaled (red) collapses to a spike.

Every step in a head is a matrix multiply, and a multiply only works when the inner dimensions match. Tracking shapes is how you catch bugs and read the design: $\mathbf{q}$ and $\mathbf{k}$ must share d_k to be dotted, and $\mathbf{W}^O$ must map back to d so heads stack.

Object	Shape	Note
input $\mathbf{x}_i$, output $\mathbf{a}_i$	$[1 \times d]$	model dimension d
$\mathbf{W}^Q, \mathbf{W}^K$	$[d \times d_k]$	project to query/key size
$\mathbf{W}^V$	$[d \times d_v]$	project to value size
$\mathbf{q}_i, \mathbf{k}_j$	$[1 \times d_k]$	dot product $\rightarrow$ scalar
$\mathbf{v}_j, \mathbf{head}_i$	$[1 \times d_v]$	weighted sum of values
$\mathbf{W}^O$	$[d_v \times d]$	reshape head back to d

Original transformer: $d = 512$, with $A = 8$ heads each of size $d_k = d_v = 64 = d/A$.

Attention for each token is **independent** of the others' outputs, so we compute the whole sequence at once. Pack the N token embeddings as rows of $\mathbf{X}$ ($[N \times d]$):

$$\mathbf{Q} = \mathbf{XW}^Q, \quad \mathbf{K} = \mathbf{XW}^K, \quad \mathbf{V} = \mathbf{XW}^V$$

- $\mathbf{Q}, \mathbf{K}$ are $[N \times d_k]$; $\mathbf{V}$ is $[N \times d_v]$.
- This is the single biggest practical advantage over RNNs: **one matrix multiply**, fully parallel on a GPU.

Stack all queries in $\mathbf{Q}$ and all keys in $\mathbf{K}$; one product $\mathbf{QK}^\top$ ($N \times N$) fills a table where entry $s_{ij} = \mathbf{q}_i \cdot \mathbf{k}_j$ is how much token i attends to token j .

- Read one **row**: it is a single query token's score against every token. A softmax over that row becomes its attention weights.
- The diagonal is each token scored against itself.
- Filling the whole table costs $O(N^2)$: attention is **quadratic** in sequence length.

		keys (looked at)		
		<i>the</i>	<i>cat</i>	<i>sat</i>
queries (looking)	<i>the</i>	s_{11}	s_{12}	s_{13}
	<i>cat</i>	s_{21}	s_{22}	s_{23}
	<i>sat</i>	s_{31}	s_{32}	s_{33}

Gold row: *sat*'s scores; a softmax over it gives how much *sat* attends to each word.

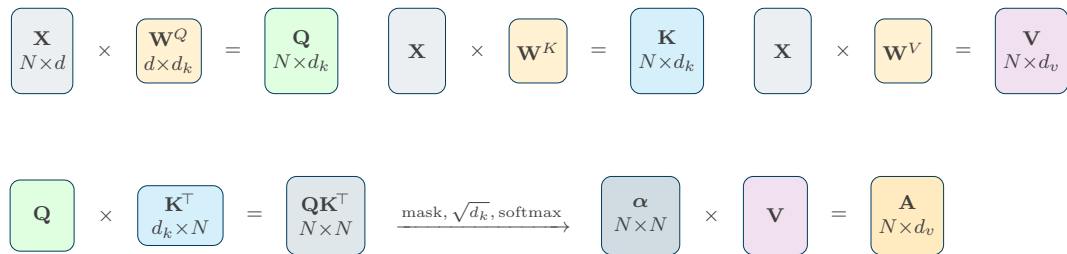
Reuse the same 3×3 grid for *the cat sat*.
To *predict* the next word, token i must not peek at tokens that come after it, or the task would be trivial.

- Before softmax, set every above-diagonal score to $-\infty$.
- $\exp(-\infty) = 0$, so those cells become 0 weight: the future is erased.
- Row *the* sees only itself; row *cat* sees *the, cat*; row *sat* sees all three.

keys $\rightarrow$		<i>the</i>	<i>cat</i>	<i>sat</i>
queries $\uparrow$	<i>the</i>	s_{11}	$-\infty$	$-\infty$
	<i>cat</i>	s_{21}	s_{22}	$-\infty$
	<i>sat</i>	s_{31}	s_{32}	s_{33}

Red cells (the future) are set to $-\infty$ before softmax, so they contribute 0.

Everything so far, batched over all N tokens at once. Read it in two rows: the **top row** builds the three projections, the **bottom row** turns them into the output.



Top: the token matrix $\mathbf{X}$ (N rows, one per token) is projected into queries, keys, values. Bottom: $\mathbf{QK}^\top$ scores every pair, the mask/scale/softmax turn scores into weights α , and $\alpha\mathbf{V}$ blends the values into the output $\mathbf{A}$.

The entire single-head self-attention reduces to two matrix multiplies around a softmax:

Scaled dot-product attention

$$\text{head} = \text{softmax}\left(\text{mask}\left(\frac{\mathbf{Q}\mathbf{K}^\top}{\sqrt{d_k}}\right)\right) \mathbf{V} \quad \mathbf{A} = \text{head} \mathbf{W}^O$$

Reading it from the inside out, each piece does one job:

- $\mathbf{Q}\mathbf{K}^\top$ dots every query with every key: the raw $N \times N$ relevance scores.
- $\div \sqrt{d_k}$ rescales so the softmax stays sensitive rather than saturating.
- mask sets future entries to $-\infty$, so token i cannot look ahead.
- softmax turns each row into weights that sum to 1; $\times \mathbf{V}$ blends the values by them.
- $\mathbf{W}^O$ projects the blend back to the model dimension d .

Take the real sentence *the cat sat* and process the verb *sat*, which should attend to its subject. Each word has already been embedded and projected; to keep the arithmetic readable we use just two interpretable feature dimensions, [thing-ness, animacy], so $d_k = d_v = 2$.

token j	role	$\mathbf{k}_j$	$\mathbf{v}_j$
<i>the</i>	determiner	(1, 0)	(1, 0)
<i>cat</i>	noun, animate	(1, 1)	(1, 1)
<i>sat</i>	verb (self)	(0, 1)	(0, 1)

Query for *sat*: $\mathbf{q} = (1, 1)$, that is, “find an animate thing.” We score it against all three keys (causal, so *sat* may see all earlier words and itself).

Step 1, raw scores $\mathbf{q} \cdot \mathbf{k}_j$ (dot product of the *sat* query with each key):

$$\underbrace{\mathbf{q} \cdot \mathbf{k}_{the} = 1}_{(1)(1)+(1)(0)}, \quad \underbrace{\mathbf{q} \cdot \mathbf{k}_{cat} = 2}_{(1)(1)+(1)(1)}, \quad \underbrace{\mathbf{q} \cdot \mathbf{k}_{sat} = 1}_{(1)(0)+(1)(1)}$$

Step 2, scale by $\sqrt{d_k} = \sqrt{2} \approx 1.41$:

$$0.71, \quad 1.41, \quad 0.71$$

Step 3, softmax (exponentiate and normalize, $\text{sum} = 2.03 + 4.10 + 2.03 = 8.16$):

$$\alpha_{the} = 0.25, \quad \alpha_{cat} = 0.50, \quad \alpha_{sat} = 0.25$$

sat puts half its attention on *cat*, its animate subject, and a quarter each on *the* and itself.

Step 4, weighted sum of value vectors:

$$\begin{aligned}\mathbf{a}_{sat} &= \alpha_{the} \mathbf{v}_{the} + \alpha_{cat} \mathbf{v}_{cat} + \alpha_{sat} \mathbf{v}_{sat} \\ &= 0.25 (1, 0) + 0.50 (1, 1) + 0.25 (0, 1) \\ &= (0.25, 0) + (0.5, 0.5) + (0, 0.25) = (0.75, 0.75)\end{aligned}$$

Read it off

The new vector for *sat* is dominated by *cat*'s value, so *sat* now carries the thing-ness and animacy of its subject. Self-attention has moved the subject's meaning into the verb through a **content-weighted average** that adapts to whatever words surround it.

A single head produces one attention distribution, which is one way of relating tokens. But language has many relationship types at once: syntax, coreference, topic. **Multi-head attention** runs A heads in parallel, each with its own $\mathbf{W}^Q, \mathbf{W}^K, \mathbf{W}^V$:

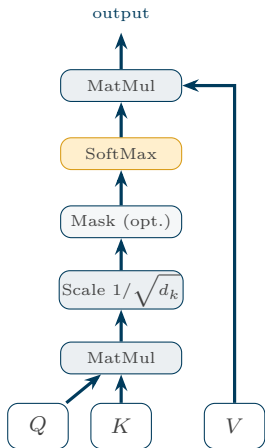
$$\mathbf{head}_c = \text{SelfAttention}(\mathbf{Q}^c, \mathbf{K}^c, \mathbf{V}^c), \quad c = 1 \dots A$$

$$\text{MultiHead}(\mathbf{X}) = (\mathbf{head}_1 \oplus \mathbf{head}_2 \oplus \dots \oplus \mathbf{head}_A) \mathbf{W}^O$$

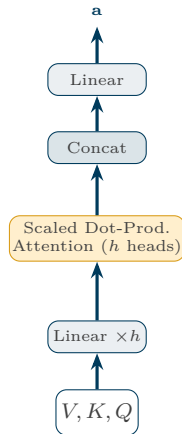
Heads are concatenated ($\oplus$), then projected back to dimension d by a single shared $\mathbf{W}^O$ that replaces the per-head projection of the single-head form. Usually $d_v = d/A$, so the concatenation is again $[N \times d]$.

Multi-Head Attention

3 Self-Attention and the Attention Head



Scaled dot-product attention



h heads run in parallel, then concat + project

Left: one head's internals. Right: h heads concatenated and mixed by $\mathbf{W}^O$ (Vaswani et al., 2017).

These labels come not from the original 2017 paper but from later probing studies that inspected trained models (Voita et al., 2019; Clark et al., 2019; Olsson et al., 2022). They report heads specializing in distinct, interpretable jobs:

- **Positional heads** attend to the previous or next token (Voita et al., 2019).
- **Syntactic heads** link a word to its syntactic partner, e.g. verb to object (Voita et al., 2019; Clark et al., 2019).
- **Coreference heads** connect a pronoun to its antecedent, as in our *it* example (Clark et al., 2019).
- **Induction heads** copy patterns seen earlier in the sequence, a basis for in-context learning (Olsson et al., 2022).

No head is told its job; specialization emerges from training.

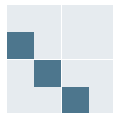
The same $N \times N$ weight matrix takes recognizable shapes across heads:

- **Diagonal** heads attend to self or nearby tokens.
- **Previous-token** heads use a shifted diagonal.
- **Broad** heads spread weight over many tokens, which averages them.

diagonal

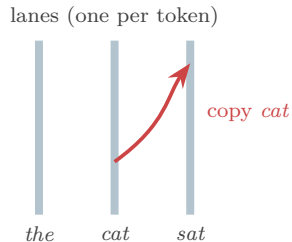


previous



A useful mental model (Elhage et al. 2021): give each token its own vertical “lane” that carries and updates its vector as it flows up through the layers.

- The feedforward layer works **inside** a lane: it never looks sideways.
- Attention is the **only** operation that draws an arrow *between* lanes, copying content from one token into another.
- In our example, attention pulls *cat* into *sat*’s lane; stacking layers lets information hop again and again.



Feedforward stays in a lane; attention moves *cat*’s content into *sat*’s lane.

The $\mathbf{QK}^\top$ matrix has N^2 entries, so attention costs $O(N^2)$ in both time and memory.

- Doubling the context **quadruples** the attention cost.
- The reason long-context models are expensive.
- Motivates flash-attention, sparse and linear attention, and the KV cache (later).

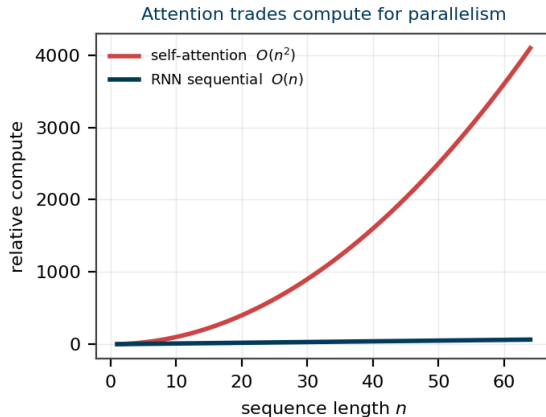




Table of Contents

4 The Transformer Block

- ▶ From Sequences to Attention
- ▶ The Attention Mechanism
- ▶ Self-Attention and the Attention Head
- ▶ **The Transformer Block**
- ▶ Architectures: GPT, BERT, and Encoder–Decoder
- ▶ Scale, Efficiency, and Limits
- ▶ A Case Study: Does Architecture Win?
- ▶ Wrap-Up

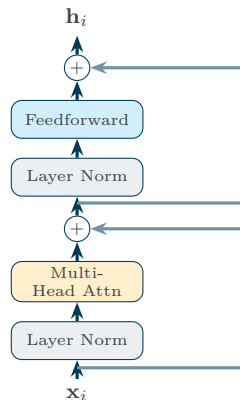
Self-attention is the core, but a **transformer block** wraps it with three more ingredients that make deep stacks trainable:

1. **Residual connections** add each sublayer's output back to its input.
2. **Layer normalization** stabilizes the scale of activations.
3. **Feedforward layer** applies per-token nonlinear processing.

We use the modern **prenorm** arrangement: normalize *before* each sublayer.

Think of each token as carrying a d -dimensional **residual stream** up through the layers.

- Each sublayer *reads* from the stream and *adds* its output back.
- The identity path lets gradients flow straight to early layers.
- This is what makes 12-, 96-, even 126-layer stacks trainable.



After mixing tokens with attention, each position is processed independently by a two-layer network with a wide hidden layer:

$$\text{FFN}(\mathbf{x}_i) = \text{ReLU}(\mathbf{x}_i \mathbf{W}_1 + \mathbf{b}_1) \mathbf{W}_2 + \mathbf{b}_2$$

$\mathbf{W}_1$ ($d \times d_{\text{ff}}$) expands the token into a wider space, ReLU adds the nonlinearity, and $\mathbf{W}_2$ ($d_{\text{ff}} \times d$) projects back to size d ; $\mathbf{b}_1, \mathbf{b}_2$ are bias vectors.

- Same weights at every position, but different from layer to layer.
- Hidden size d_{ff} is usually $4\times$ the model dimension (e.g. $d = 512$, $d_{\text{ff}} = 2048$).
- This is where much of the model's parameters, and stored knowledge, live.

In a deep residual stack, activation magnitudes drift and can blow up. Layer norm resets each token to a controlled scale: it is a per-token **z-score** over the d components of that token's vector.

$$\mu = \frac{1}{d} \sum_{j=1}^d x_j, \quad \sigma = \sqrt{\frac{1}{d} \sum_{j=1}^d (x_j - \mu)^2}$$

$$\hat{\mathbf{x}} = \frac{\mathbf{x} - \mu}{\sigma}, \quad \text{LN}(\mathbf{x}) = \gamma \hat{\mathbf{x}} + \beta$$

- Output has zero mean, unit variance, keeping activations in a stable range.
- Learned gain γ and offset β restore flexibility.
- Unlike batch norm, it does **not** depend on other examples, which makes it ideal for variable-length text.
- Introduced by Ba, Kiros & Hinton (2016).

With $\mathbf{X}$ the block input ($[N \times d]$), the prenorm transformer layer is:

$$\mathbf{O} = \mathbf{X} + \text{MultiHead}(\text{LayerNorm}(\mathbf{X}))$$

$$\mathbf{H} = \mathbf{O} + \text{FFN}(\text{LayerNorm}(\mathbf{O}))$$

- Two sublayers, each: normalize $\rightarrow$ transform $\rightarrow$ add to the residual stream.
- Input and output are both $[N \times d]$, so blocks **stack** cleanly.

Postnorm (original 2017): normalize *after* adding the sublayer.

$$\mathbf{h} = \text{LayerNorm}(\mathbf{x} + \text{Sublayer}(\mathbf{x}))$$

Prenorm (modern default): normalize *before* the sublayer.

$$\mathbf{h} = \mathbf{x} + \text{Sublayer}(\text{LayerNorm}(\mathbf{x}))$$

- Prenorm keeps a clean identity path through the residual stream.
- Gradients reach early layers more reliably; training is more stable, often without learning-rate warmup.
- Deep stacks (dozens of layers) almost always use prenorm.

Two ways to grow a model, with different effects:

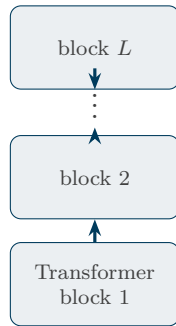
- **Depth** (n_{layer}): more blocks allow more rounds of token-mixing and more abstract, composed representations.
- **Width** (d, d_{ff}): larger vectors hold more information per token, and parameters grow as d^2 .

Real models scale both together. Because parameters grow with d^2 but only linearly with depth, width is an expensive lever, which is one reason scaling laws track total parameter count rather than depth alone.

Every block takes an $[N \times d]$ matrix and returns one of the *same* shape, so blocks snap together like identical Lego bricks. Each one **refines** the representation a little more:

- The output $\mathbf{H}$ of one block is the input $\mathbf{X}$ of the next, no reshaping needed.
- Early layers capture local syntax; later layers capture meaning and long-range structure.
- Typical depths: 12 (BERT-base, GPT-2 small) to 96 (GPT-3) to 126 (Llama-3 405B).
- A final extra layer norm sits on top before the output head.

to output head



$\mathbf{X}$ (embeddings)

Before embedding, text is split into tokens. Modern transformers use **subword** units (BPE, WordPiece, SentencePiece) rather than whole words:

- **Byte-Pair Encoding** starts from characters and repeatedly merges the most frequent adjacent pair.
- Common words become single tokens; rare words split into pieces (*tokenization* → *token, ization*).
- No out-of-vocabulary problem: any string is representable.
- A fixed vocabulary (e.g. 32K–128K) balances sequence length against table size.

Before any block, each token id is mapped to a vector via the **embedding matrix $\mathbf{E}$** of shape $[|V| \times d]$:

- Tokenize text (BPE / SentencePiece) into ids, e.g. *thanks for all the* $\rightarrow [5, 4000, 10532, 2224]$.
- Row i of $\mathbf{E}$ is the embedding of token i .
- Equivalently: multiply a one-hot row vector by $\mathbf{E}$ to select that row.

Stacking N token embeddings gives an $[N \times d]$ matrix, but it has **no notion of position** yet.

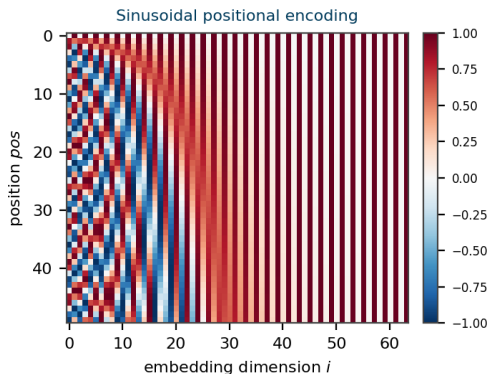
Self-attention is **permutation-equivariant**: it treats the input as a *set*. Shuffle the tokens and the attention outputs shuffle identically, so order information is lost.

“dog bites man” and *“man bites dog”* would look the same to bare attention.

Fix: add a **positional embedding** to each token embedding, so the input encodes both *what* the token is and *where* it sits.

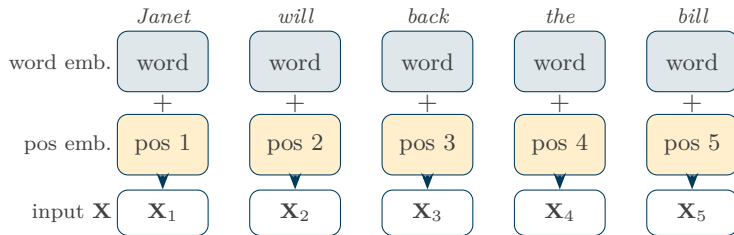
A positional encoding is a length- d vector, one per slot, added to the word embedding to stamp in order. Two choices:

- **Learned absolute.** A trainable vector per position; what BERT and GPT use.
- **Sinusoidal** (Vaswani et al., 2017): fixed waves, $\mathbf{P}[pos, 2i] = \sin(pos/10000^{2i/d})$ (odd dims use cos).
- Low dims oscillate fast, high dims slowly, like clock digits, so every position gets a distinct pattern that extends to unseen lengths.



Read a row: one position's signature across the d dimensions. Left columns wiggle fast, right columns slow; no two rows match.

The matrix $\mathbf{X}$ that enters the first block is the sum of two embeddings per position (example sentence *Janet will back the bill*). Each column adds straight down:

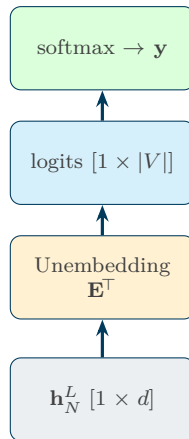


Word embedding + positional embedding $\rightarrow$ composite input of the same dimension d .

After the final block, the top token's vector $\mathbf{h}_N^L$ is turned into a distribution over the vocabulary. Since row t of $\mathbf{E}$ is token t 's embedding, the dot product $\mathbf{h}_N^L \cdot \mathbf{E}_t$ scores how well the output matches each token:

$$\mathbf{u} = \mathbf{h}_N^L \mathbf{E}^\top, \quad \mathbf{y} = \text{softmax}(\mathbf{u})$$

- The **unembedding** $\mathbf{E}^\top$ maps $[1 \times d] \rightarrow [1 \times |V|]$ logits.
- **Weight tying**: reuse $\mathbf{E}$ as the unembedding.
- Softmax gives $P(\text{next token} \mid \text{context})$.



End to end, a causal language model maps an input token w_i to a prediction of the next token w_{i+1} :

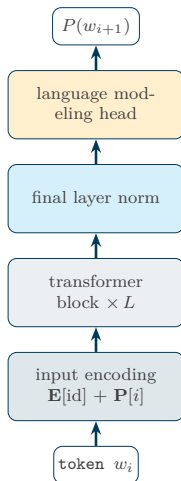




Table of Contents

5 Architectures: GPT, BERT, and Encoder–Decoder

- ▶ From Sequences to Attention
- ▶ The Attention Mechanism
- ▶ Self-Attention and the Attention Head
- ▶ The Transformer Block
- ▶ Architectures: GPT, BERT, and Encoder–Decoder
- ▶ Scale, Efficiency, and Limits
- ▶ A Case Study: Does Architecture Win?
- ▶ Wrap-Up

The same block supports three architecture families, differing in **what each token may attend to**:

Family	Attention	Example
Decoder-only	causal (past only)	GPT
Encoder-only	bidirectional (all tokens)	BERT
Encoder–decoder	encoder bidir. + decoder cross	T5, translation

The masking pattern is the main thing that changes.

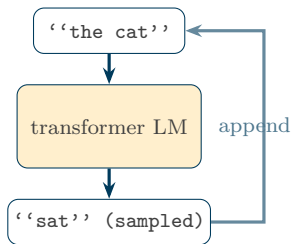
GPT (Radford et al.) is a stack of **causal** transformer blocks trained on plain next-token prediction:

- Each token attends only to itself and earlier tokens (the mask we saw).
- Objective: maximize $P(w_t \mid w_{<t})$, the next word w_t given all preceding words $w_{<t}$, over a huge corpus.
- One simple objective, scaled up, yields a general-purpose generator.

This left-to-right, autoregressive setup is the basis of modern large language models.

Generation is a loop: predict the next token, append it, feed it back in.

1. Run the stack on the current context.
2. Read $P(\text{next})$ from the head.
3. **Sample** a token from that distribution.
4. Append and repeat.



How we pick from $P(\text{next})$ trades off **quality** vs. **diversity**:

- **Greedy**: always take the argmax, which is coherent but repetitive.
- **Top- k** : keep the k most probable tokens, renormalize, then sample.
- **Top- p (nucleus)**: keep the smallest set whose probability mass exceeds p , which adapts to how peaked the distribution is (Holtzman et al., 2020).
- **Temperature**: rescale logits to sharpen (< 1 , more deterministic) or flatten (> 1 , more random) the distribution.

A striking property of large decoder-only models: they perform new tasks from a **prompt alone**, with no weight updates.

- **Zero-shot**: describe the task in the prompt.
- **Few-shot**: include a handful of input/output examples before the query.
- The model infers the pattern within its context window and continues it.

This behavior strengthens sharply with scale and is thought to rely partly on **induction heads**: attention heads that detect a repeated pattern earlier in the context and copy its continuation forward.

BERT (Devlin, Chang, Lee & Toutanova, 2019) uses **bidirectional** attention: every token sees the whole sentence, left and right.

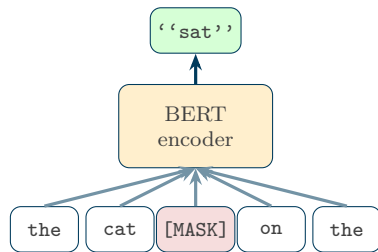
- No causal mask, which makes it ideal for *understanding* rather than generation.
- Produces a rich contextual embedding for each token.
- Trained with **masked language modeling** instead of next-token prediction.

Seeing both directions is what lets BERT excel at classification and tagging, but it is also why BERT cannot generate text left to right the way GPT does.

BERT randomly **masks** about 15% of tokens and trains the model to reconstruct them from both sides:

the cat [MASK] on the mat → predict *sat*

- Bidirectional context makes the task informative.
- A “fill in the blank” objective forces deep understanding.



Both use the same block; they differ only in which cells of the $N \times N$ attention matrix are allowed. Rows are queries, columns are keys; a mark means “row may attend to column.”

GPT (causal)

k_1 k_2 k_3 k_4

q_1	✓	×	×	×
q_2	✓	✓	×	×
q_3	✓	✓	✓	×
q_4	✓	✓	✓	✓

BERT (bidirectional)

k_1 k_2 k_3 k_4

q_1	✓	✓	✓	✓
q_2	✓	✓	✓	✓
q_3	✓	✓	✓	✓
q_4	✓	✓	✓	✓

Left: a query may see only itself and the past, so the future (×) is blocked, one word at a time. Right: every query attends everywhere, so BERT reads the whole sentence at once, ideal for understanding but not for left-to-right generation.

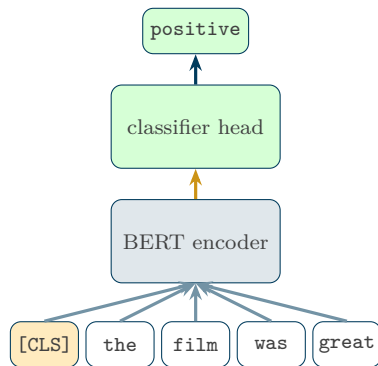
The paradigm that reshaped NLP:



- Pretraining learns general language structure once, at great expense.
- Fine-tuning adapts cheaply to sentiment, QA, NER, and more.
- The text analogue of transfer learning from pretrained vision backbones.

BERT prepends a special [CLS] token to every input. Its final-layer vector serves as a sentence summary:

- Add a small classifier head on top of the [CLS] vector.
- Fine-tune the whole model on labeled examples (e.g. sentiment).
- The same recipe, swapping the head, covers entailment, QA, and NER.



The [CLS] vector feeds the task head.

In practice you rarely build a transformer from scratch. The **Hugging Face** ecosystem supplies pretrained models and a uniform fine-tuning API:

- **transformers**: thousands of checkpoints (BERT, GPT-2, T5, Llama) behind one API; `AutoModel` and `AutoTokenizer` load any of them by name.
- **datasets**: one-line access to benchmark corpora; the `Trainer` class runs the fine-tuning loop for you.
- **PEFT / LoRA**: adapt a large model by training a few small extra matrices, so fine-tuning fits on a single GPU.
- The **Hub** hosts the models, datasets, and interactive demos (Spaces).

```
from transformers import pipeline
clf = pipeline("sentiment-analysis")
clf("Attention is a beautiful idea")    # -> label: POSITIVE
```

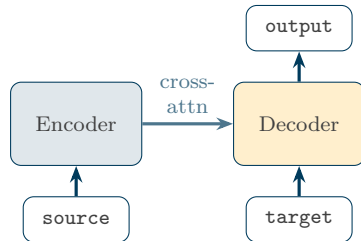
Recommended, no-cost resources to go from these slides to running code:

- **Hugging Face NLP Course** (huggingface.co/learn): free, end-to-end, covers tokenizers, `Trainer`, and PEFT.
- **Transformers documentation**: task guides for classification, token classification, QA, and summarization.
- **PEFT library docs**: LoRA and QLoRA recipes for adapting large models cheaply.
- **Model and dataset cards on the Hub**: each lists intended use, training data, and known limitations, worth reading before you fine-tune.

The companion notebook applies this workflow to a real dataset.

For sequence-to-sequence tasks (translation, summarization), the **encoder–decoder** keeps both halves:

- Encoder: bidirectional self-attention over the source.
- Decoder: causal self-attention over the target so far.
- **Cross-attention**: decoder queries attend to encoder keys and values, which is the modern version of Bahdanau attention.



The math is identical; only the source of keys and values changes.

	Self-attention	Cross-attention
Query $\mathbf{Q}$ from	current sequence	decoder (target)
Key/Value $\mathbf{K}, \mathbf{V}$ from	current sequence	encoder (source)
Purpose	mix within a sequence	inject source into target

Cross-attention is how a translation decoder “looks at” the source sentence while generating.



Table of Contents

6 Scale, Efficiency, and Limits

- ▶ From Sequences to Attention
- ▶ The Attention Mechanism
- ▶ Self-Attention and the Attention Head
- ▶ The Transformer Block
- ▶ Architectures: GPT, BERT, and Encoder–Decoder
- ▶ **Scale, Efficiency, and Limits**
- ▶ A Case Study: Does Architecture Win?
- ▶ Wrap-Up

Transformer language models train with **cross-entropy** (negative log-likelihood) on next-token prediction:

$$L = -\frac{1}{T} \sum_{t=1}^T \log P(w_t \mid w_{<t})$$

Here T is the sequence length, w_t the token that actually came next at position t , and $w_{<t}$ everything before it. The loss is small only when the model puts *high* probability on the true next token, so minimizing it teaches the model to predict text.

- Every position contributes a term, and all T are scored **in parallel**.
- No labels needed, because the text *is* the supervision (self-supervised).
- This single objective, plus scale, produces today's capabilities.

Large language models are large. *Llama 3.1 405B* has 405B parameters, $L=126$ layers, $d=16,384$, $A=128$ heads, trained on 15.6T tokens. Performance is driven mainly by three factors:

- **Model size:** number of parameters.
- **Data size:** number of training tokens.
- **Compute:** total training operations.

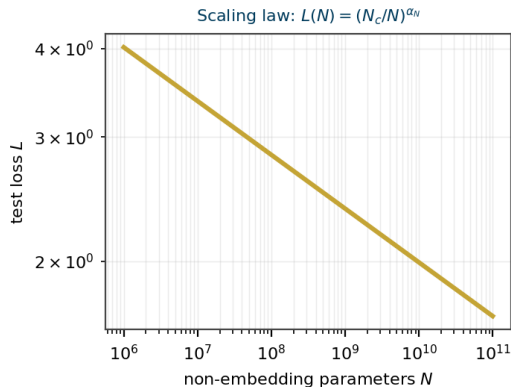
How these relate is captured by **scaling laws**.

Test loss falls as a **power law** in each factor when the others are held fixed (Kaplan et al., 2020):

$$L(N) = \left(\frac{N_c}{N} \right)^{\alpha_N}$$

N is the parameter count, N_c a fitted constant (the scale where the fit reaches unit loss), and $\alpha_N \approx 0.076$ the small exponent.

- Straight but gently sloped line on a log-log plot.
- Lets you forecast a bigger model's loss before training it.
- Smooth but **diminishing**: each further drop costs more.



Ignoring biases and embeddings, a transformer's parameter count is dominated by the blocks:

$$N \approx 12 n_{\text{layer}} d^2 \quad (\text{assuming } d_{\text{attn}} = d_{\text{ff}}/4 = d)$$

- Each block contributes $\sim 12d^2$: four attention projections ($\mathbf{W}^Q, \mathbf{W}^K, \mathbf{W}^V, \mathbf{W}^O$) at d^2 each give $4d^2$, and the two FFN matrices ($d \times 4d$ and $4d \times d$) add $8d^2$.
- GPT-3: $n_{\text{layer}} = 96, d = 12,288 \Rightarrow 12 \cdot 96 \cdot 12288^2 \approx 174$ billion; embeddings bring the reported total to 175B.
- Parameters scale with the **square** of the model dimension.

Given a fixed compute budget, should you build a *bigger* model or train on *more* data?

- Early large models were **undertrained**, carrying too many parameters for too few tokens (Hoffmann et al., 2022, “Chinchilla”).
- Compute-optimal training scales parameters and tokens **together**, roughly in proportion (about 20 tokens per parameter).
- A smaller model trained on more data beat a larger one at the same compute.

Scaling laws do more than describe behavior; they guide how to spend a training budget.

Kaplan's law varies one factor at a time. Hoffmann et al. (2022) instead fit the loss as a function of *both* parameters N and training tokens D together:

$$L(N, D) = \underbrace{E}_{\text{irreducible}} + \frac{A}{N^\alpha} + \frac{B}{D^\beta}$$

E is the entropy of natural text you can never beat; the two terms shrink as you add parameters or data. Minimizing L under a compute budget $C \approx 6ND$ gives the key result:

- Optimal N and D both grow like $\sqrt{C}$, so **scale them equally**: roughly **20 training tokens per parameter**.
- Proof by build: Chinchilla (70B params, 1.4T tokens) beat Gopher (280B, 300B tokens) at the *same* compute, across benchmarks.
- Lesson: many earlier models were oversized and starved of data, wasting compute on parameters they could not train.

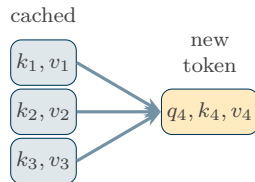
Some capabilities are nearly absent in small models and then appear, sometimes abruptly, as scale grows:

- Multi-step arithmetic, instruction following, chain-of-thought reasoning (Wei et al., 2022).
- Smooth loss curves can hide sharp jumps in task accuracy.

Whether these jumps are genuine phase changes or artifacts of how we measure them is actively debated. Either way, scale unlocks behaviors not designed into the architecture.

At inference we generate one token at a time. Recomputing keys and values for all prior tokens at each step is wasteful, since they never change.

- **Cache** the **K** and **V** of past tokens.
- Each new token computes only its own **q**, **k**, **v** and reads the rest from cache.
- Turns per-step cost from $O(N^2)$ back toward $O(N)$.
- Cost: cache memory grows with context length.



Fine-tuning all parameters of a billion-parameter model is costly. **PEFT** methods adapt a frozen model by training a tiny set of extra weights:

- **LoRA**: inject low-rank update matrices into attention projections and train only those.
- **Adapters and prompt tuning**: small modules or learned prompt vectors.
- Often $< 1\%$ of parameters trained, with near full-fine-tune quality.

This makes adapting large models feasible on modest hardware.

- **Quadratic context.** Attention cost grows with N^2 ; very long documents remain hard.
- **Hallucination.** A fluent next-token predictor can state false things confidently.
- **Data and energy hunger.** Pretraining consumes enormous corpora and compute.
- **Bias and opacity.** Models absorb biases from training text, and treating scale alone as understanding has been cautioned against (Bender et al., 2021).



Table of Contents

7 A Case Study: Does Architecture Win?

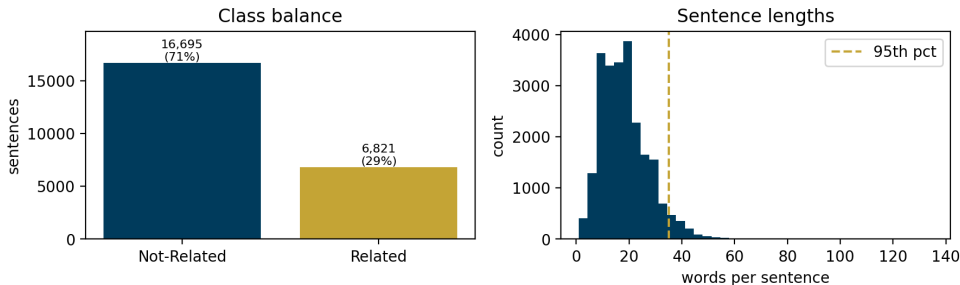
- ▶ From Sequences to Attention
- ▶ The Attention Mechanism
- ▶ Self-Attention and the Attention Head
- ▶ The Transformer Block
- ▶ Architectures: GPT, BERT, and Encoder–Decoder
- ▶ Scale, Efficiency, and Limits
- ▶ A Case Study: Does Architecture Win?
- ▶ Wrap-Up

A patient starts a new medication and days later develops a severe rash. Is the drug to blame, or is it a coincidence? Now ask that question across *millions* of case reports, clinical notes, and journal articles.

- **Pharmacovigilance** is the science of detecting, assessing, and preventing adverse effects of drugs once they reach the market (WHO). Clinical trials are too small and too short to catch rare or long-term reactions.
- Adverse drug events are a leading cause of emergency visits and hospital admissions, yet the evidence is scattered across *unstructured text* and is chronically under-reported.
- Manual review cannot keep pace with the literature, so drug-safety teams need to **read text at scale** and flag the sentences worth a human's attention.

The task. Read one sentence from a case report and decide: does it describe a drug-related *adverse effect* (*Related*) or not (*Not-Related*)? A model that does this well surfaces safety signals humans would miss.

ADE Corpus v2: exploratory data analysis



ADE Corpus v2 (Gurulingappa et al., 2012): 23,516 sentences from MEDLINE case reports, hand-annotated. **Imbalanced** (29% *Related*) and **short** (median 17 words) — both facts shape the modeling choices below.

Same data, same 80/20 split, three learners scored on a held-out 4,704 sentences:

- **TF-IDF + Naive Bayes** — the classical baseline you should always run first.
- **Bidirectional LSTM** — recurrence, sequential structure built in.
- **From-scratch Transformer encoder** — the block we just built (embed 64, 2 heads, 1 layer).

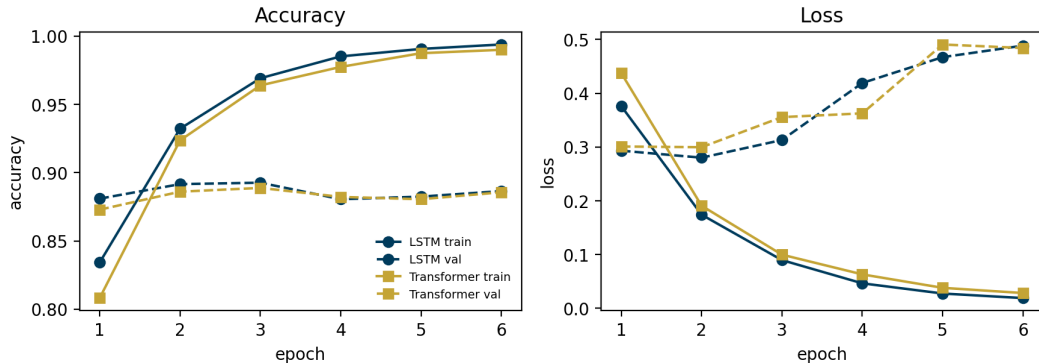
Each model trains up to 10 epochs with **early stopping** (restore best validation weights), so the run itself answers “would more epochs help?” with evidence. **The question:** does the Transformer win *just by being a Transformer*?

Best validation weights, held-out 4,704 sentences:

Model	Val. accuracy	F1 (<i>Related</i>)
TF-IDF + Naive Bayes	0.837	0.624
Bidirectional LSTM	0.893	0.808
From-scratch Transformer	0.889	0.803

The neural models clearly beat classical Naive Bayes. The Transformer and the LSTM are **effectively tied** (0.889 vs. 0.893): the Transformer matches the LSTM without pulling ahead of it.

Training vs. validation: LSTM and from-scratch Transformer

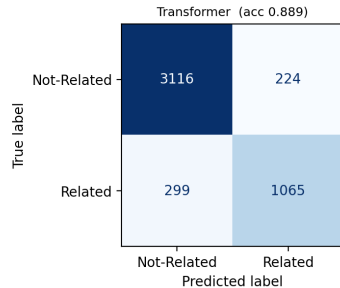
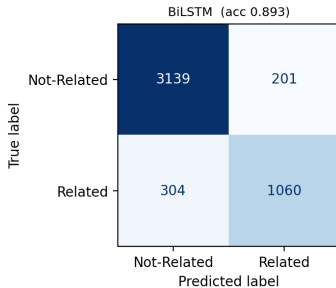
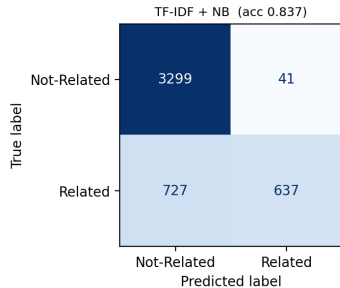


Both models climb past $\sim 99\%$ *training* accuracy but stall near 89% on validation: textbook overfitting, even on $\sim 19k$ short sentences. Validation accuracy peaks by epoch 2–3 and then drifts down, which is why **more epochs do not help** and early stopping keeps the best weights. The fix is more data or pretraining, because additional 98 / 105

Where the Errors Hide

7 A Case Study: Does Architecture Win?

Confusion matrices on the validation set



Accuracy hides the minority class. Naive Bayes catches only 637 of 1,364 *Related* cases; the LSTM recovers 1,060 and the Transformer 1,065. For pharmacovigilance a *missed* adverse event is the costly error, so recall and F1 on the positive class matter more than raw accuracy.

- **Transformers are data-hungry.** Self-attention has weak inductive bias: it must *learn* word order and locality from data, while an LSTM has sequential structure built in. Even $\sim 19\text{k}$ sentences do not fully pay for that flexibility.
- **Short inputs blunt the advantage.** Attention shines on long-range dependencies; here the median sentence is 17 words, so range is not the bottleneck.
- **No pretraining.** The famous wins come from pretraining on billions of tokens, then fine-tuning. Our encoder starts from random weights.
- **Strong baselines are the honest bar.** TF-IDF + Naive Bayes hits 0.84 in under a second; always run it first.

This is the scaling-law story in miniature: Transformers earn their dominance through **scale and pretraining**. At this size their edge over the LSTM is *parallelism*, processing the whole sequence at once while matching it on accuracy.



Table of Contents

8 Wrap-Up

- ▶ From Sequences to Attention
- ▶ The Attention Mechanism
- ▶ Self-Attention and the Attention Head
- ▶ The Transformer Block
- ▶ Architectures: GPT, BERT, and Encoder–Decoder
- ▶ Scale, Efficiency, and Limits
- ▶ A Case Study: Does Architecture Win?
- ▶ **Wrap-Up**

Each family captures a different piece of the puzzle. The transformer is the first to get **both at once, in parallel**: word order and cross-word context.

	Word-order awareness	Context awareness (cross-word)
Bag-of-unigrams	No	No
Bag-of-bigrams	Very limited	No
RNN	Yes	No
Self-attention	No	Yes
Transformer	Yes	Yes

1. **Attention replaced recurrence:** direct, parallel, constant-path-length token interactions.
2. **One equation does the work:** $\text{softmax}(\text{mask}(\mathbf{Q}\mathbf{K}^\top / \sqrt{d_k}))\mathbf{V}$, with multiple heads.
3. **A block** wraps attention with residuals, layer norm, and a feedforward layer; stack many.
4. **Position is added explicitly** because attention treats its input as an unordered set of tokens.
5. **One block, three families:** GPT (causal), BERT (bidirectional), encoder–decoder (cross-attention).
6. **Scale follows power laws;** efficiency tricks (KV cache, PEFT) make it practical.

- Vaswani, A., Shazeer, N., Parmar, N., Uszkoreit, J., Jones, L., Gomez, A., Kaiser, Ł., Polosukhin, I. (2017). Attention Is All You Need. *NeurIPS 2017*.
- Bahdanau, D., Cho, K., Bengio, Y. (2015). Neural Machine Translation by Jointly Learning to Align and Translate. *ICLR 2015*.
- Devlin, J., Chang, M.-W., Lee, K., Toutanova, K. (2019). BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding. *NAACL-HLT 2019*.
- Hochreiter, S., Schmidhuber, J. (1997). Long Short-Term Memory. *Neural Computation*, 9(8).
- Ba, J. L., Kiros, J. R., Hinton, G. E. (2016). Layer Normalization. *arXiv:1607.06450*.
- Kaplan, J., McCandlish, S., Henighan, T., Brown, T., et al. (2020). Scaling Laws for Neural Language Models. *arXiv:2001.08361*.
- Hoffmann, J., Borgeaud, S., Mensch, A., et al. (2022). Training Compute-Optimal Large Language Models (Chinchilla). *NeurIPS 2022*.
- Holtzman, A., Buys, J., Du, L., Forbes, M., Choi, Y. (2020). The Curious Case of Neural Text Degeneration. *ICLR 2020*.

- Voita, E., Talbot, D., Moiseev, F., Sennrich, R., Titov, I. (2019). Analyzing Multi-Head Self-Attention. *ACL 2019*.
- Clark, K., Khandelwal, U., Levy, O., Manning, C. D. (2019). What Does BERT Look At? An Analysis of BERT's Attention. *BlackboxNLP @ ACL 2019*.
- Elhage, N., Nanda, N., Olsson, C., et al. (2021). A Mathematical Framework for Transformer Circuits. *Anthropic, Transformer Circuits Thread*.
- Olsson, C., Elhage, N., Nanda, N., et al. (2022). In-context Learning and Induction Heads. *Anthropic, Transformer Circuits Thread*.
- Wei, J., Tay, Y., Bommasani, R., et al. (2022). Emergent Abilities of Large Language Models. *TMLR 2022*.
- Rogers, A., Kovaleva, O., Rumshisky, A. (2020). A Primer in BERTology. *TACL*, 8.
- Bender, E. M., Gebru, T., McMillan-Major, A., Shmitchell, S. (2021). On the Dangers of Stochastic Parrots. *FAccT 2021*.
- Gurulingappa, H., Rajput, A. M., Roberts, A., et al. (2012). Development of a Benchmark Corpus to Support the Automatic Extraction of Drug-Related Adverse Effects from Medical Case Reports. *Journal of Biomedical Informatics*, 45(5). (ADE Corpus v2)
- Jurafsky, D., Martin, J. H. *Speech and Language Processing* (3rd ed.), Ch. 8. Chollet, F. *Deep Learning with Python* (2nd ed.), Ch. 11.